

Learning Guide — OS-for-AI

Who is this for? Anyone discovering this project who wants to understand *everything*, taking nothing for granted. We start from zero. Every technical word is defined the first time it appears, and every decision is justified — not merely stated.

How to read it? In order. Each section builds on the previous one. The > 💡 boxes offer an analogy or a plain restatement. The > ⚠ boxes flag a pitfall or a nuance that is often confused.

A note on vocabulary. This is the English edition of the project's learning guide (the French original is in docs/guide-apprentissage.md). The project's code and specification are written in English, so the technical terms here match the source files directly.

💡 **In one sentence:** this project asks what an operating system would look like if its primary users were not humans but autonomous AI programs — and then it builds and tests a real part of one.

Table of contents

1. [The basic words: operating system, kernel, agent](#)
 2. [The problem: why Linux does not fit AI agents](#)
 3. [The target user: "profile B"](#)
 4. [The project's vocabulary, defined once and for all](#)
 5. [The thesis: three \(then six\) measurable properties](#)
 6. [The architecture: the pieces and how they fit](#)
 7. [The journey of one action, end to end](#)
 8. [The five architectural bets, and why](#)
 9. [The two substrates: Linux, then seL4](#)
 10. [The method: how we establish that it is true](#)
 11. [What is proven, what is not](#)
 12. [Pocket glossary](#)
-

1. The basic words

Before discussing the project, three words are needed.

An operating system (OS)

An **operating system** (OS) is the program that sits between the hardware (processor, memory, disk, network card) and the programs you run. When a word processor wants to save a file, it does not talk to the hard disk directly: it asks the operating system to do it. The OS arbitrates, protects, and shares resources among all programs.

Examples: **Linux**, **Windows**, **macOS**.

💡 Think of the operating system as a building manager. The tenants (the programs) never touch the plumbing or the electrical meter directly. They go through the manager, who decides who has access to what.

The kernel

The **kernel** is the heart of the operating system: the part with full power over the hardware. It decides which program runs at which moment, which memory region is readable by whom, and so on. When people say "Linux," they technically mean mostly its kernel.

⚠️ **Key point for this project:** this project **does not write a kernel**. It builds a runtime (see below) that installs **on top of** an existing kernel (Linux, or seL4). It is not an operating system in the "it boots the machine" sense — it is an operating system in the "it provides programs with the services an OS provides" sense.

A runtime

A **runtime** is a program that hosts and runs other programs, providing them with an environment and services. This project's runtime hosts AI agents and offers them "OS-class" services (memory, traceability, access control) without itself being the machine's kernel.

💡 From an agent's point of view, this runtime *is* its operating system: the agent never sees Linux directly, only the abstractions the runtime presents.

An AI agent

In this project, an **agent** is a program driven by an AI — typically a Large Language Model (LLM, like those behind ChatGPT or Claude) — that performs tasks **autonomously**: it receives a goal, reasons, acts on the system, observes the result, and repeats — without a human validating each step.

Concrete examples that exist today: Claude Code, Devin, SWE-agent. They run for hours, execute thousands of operations, with minimal human supervision.

2. The problem

Today's operating systems were designed between 1960 and 1990, for humans sitting at a terminal. These assumptions were never written down — they seemed self-evident. They no longer are once the user is an AI agent.

Here are the seven implicit assumptions, and what they cost an agent.

A-1 — "The user perceives time in milliseconds"

Linux is tuned so that a human *feels* the system as responsive: it slices processor time into chunks of a few milliseconds to create the illusion that everything runs at once.

Cost for an agent: running 1,000 agents at once, each as a classic program, forces the system to spend a huge share of its time juggling between them. At 1,000 agents this juggling overhead can consume a whole processor core — producing nothing useful.

A-2 — "The interface is text read by a human"

Under Unix, "everything is a file" and everything is read as text. The `ls` command lists files as text; `ps` lists programs as text. This text is meant for a human eye, not for machine analysis.

Cost for an agent: the agent must *parse* this text. But the format changes across system versions, configured language, encoding... It is fragile, and it is a whole class of avoidable bugs.

A-3 — "System state accumulates implicitly"

There is no command that says "give me a complete, consistent photograph of the system's state right now." State is scattered: modified files, program memory, open network connections... A human mentally reconstructs what happened.

Cost for an agent: it is impossible to take a clean "photograph" (a *snapshot*) of its state in order to return to it later. Therefore it is impossible to cleanly **undo** a mistake. (This is the subject of property P2.)

A-4 — "Trust is tied to identity"

Under Unix, rights depend on *who you are* (the user identity). The "root" identity (the administrator) grants all rights, all the time, over everything.

Cost for an agent: an AI agent does not have a stable, predictable identity — its behavior depends on what it is asked and on the model's share of randomness. Granting it "all rights because we trust it" amounts to handing the house keys to an entity whose next-minute behavior is unknown. And there is no *native* way to say "here is the right to read this exact file, only for five minutes, revocable at any time."

A-5 — "Parallelism is bounded by what a human can supervise"

System tools display a few dozen to a few hundred programs — what a human can monitor. Each program carries a fixed cost: memory reserved merely to exist.

Cost for an agent: on a 16 GB machine, one Docker container per agent (the common practice, see below) costs 100 MB to 1 GB *each*. The result: a few dozen agents, not thousands. The cost is not physical — it comes from abstractions designed for small numbers.

 **Docker / container:** a technology that wraps a program with everything it needs (libraries, files) into an isolated "box." Convenient, but heavy: each box embeds a copy of many things.

A-6 — "Persistence is mutable and historyless"

When you overwrite a file, the old version is lost. The file system keeps no history by default.

Cost for an agent: each agent must reinvent its own save/restore logic. This is duplicated work, inconsistent from one agent to another, and unguaranteed: a failure at the wrong moment can leave a half-written state.

A-7 — "Observability is reconstructive"

To understand *why* something happened on Linux, you must cross-reference several separate logs, by hand, after the fact. This is slow and incomplete.

Cost for an agent: for an agent that has executed ten million operations, recovering the causal chain of one specific operation is not a fast, on-demand operation. Yet auditing an autonomous agent is exactly that.

The signal that confirms the problem

The proof that something is missing at the OS level? **Developers reinvent it at the application level.**

Tools like **Temporal** or **LangGraph** re-implement, at great cost, inside each application, functions that *should* be system services: undo, deterministic replay, explicit state management.

💡 This is a classic pattern in computing: a feature is first bolted onto each application, then eventually descends into the system once people understand that this is *its* place. Memory garbage collection, embedded databases, containers: all followed this path.

The project's question: which OS-level abstractions would make this hand-rolled layer disappear?

3. Profile B

One cannot design for "AI agents" in general — they are too varied. The project therefore picks a precise target, called **profile B**:

Criterion	Value
(a) Lifetime	from 1 hour to 1 month — neither ephemeral nor eternal
(b) State	persistent between actions (it does not restart from scratch each time)
(c) Volume	from 10^4 to 10^8 actions over its whole life
(d) Supervision	a human intervenes <i>occasionally</i> (every hour/day), not on every action
(e) Delegation	the agent can create sub-agents with a subset of its rights

💡 Profile B differs from three other use cases: it is neither a *batch* agent (which lives a few seconds and forgets everything), nor a *permanent service* (which runs indefinitely without checkpoints), nor an *interactive* agent (where a human validates every step).

An important honesty: the measuring instrument is part of the cage

Large language models reason *like* humans: slowly, in language, at a rate of a few actions per second at most. The project is aware of this and says so clearly: by calibrating everything on these models, one risks designing an OS "for humans who never sleep" rather than for genuinely different agents — a reinforcement-learning agent emits a *million* actions per second, and the project's thresholds then make no sense.

The project therefore separates: - **What is invariant** (true for any supervised autonomous agent): you need a history of causes, a way to undo, and revocable delegation of rights. - **What is sized for**

today's models: the numeric thresholds (10 ms, 100 ms, the 1 h – 1 month corridor). To be revised when other kinds of agents dominate.

💡 Why test on large language models, then? Because they are **slow and verbose**: they verbalize their reasoning, so when a design flaw exists, they fail in a *visible and understandable* way. They are poor subjects for measuring performance, but excellent subjects for revealing design flaws.

4. Vocabulary

These terms appear everywhere. They are worth assimilating: the rest of the document uses them without redefining them.

Action

The **basic unit** of the project. An action = a message received and processed by an agent, **or** a message emitted by an agent, as observed by the runtime.

⚠️ An action is *not* a machine instruction, nor a function call, nor a network request. It is a **message exchange between actors**. A purely internal computation, with no message, is not an action here — a deliberate choice: we observe exchanges, not every internal cog.

Actor / Agent

An **actor** is an entity that receives messages, processes them one by one, then emits messages. An **agent** is an actor identified by a permanent identifier (`agent_id`).

Key point: **an agent is internally single-tasked**. It has no parallel threads. To do several things at once, it **spawns** sub-agents. Concurrency happens *between* agents, never *inside* an agent.

💡 Why this choice? Because internal concurrency (several threads sharing the same memory) is the main source of bugs that cannot be reproduced. By forbidding it, each agent becomes deterministic and replayable (see P5).

Another key point: a failure followed by a restart **does not create a new agent**. The same `agent_id` resumes from its last saved state. An agent's identity is not its process number (PID) — it is its `agent_id`, which survives restarts.

Capability

A **capability** is a **token granting a precise right**. It is the heart of the project's security model. It has

four properties:

- **Non-ambient**: no rights by default. To perform an operation, the actor must *hold* the token that authorizes it. No token = no access, period.
- **Delegable**: an actor can pass a token (or a reduced version) to another actor.
- **Derivable / attenuable**: one can create a *more restricted* version of a token (fewer rights, narrower scope). **Never the reverse**. A sub-agent can never hold more rights than its parent.
- **Revocable**: a token can be withdrawn at any time; this **cascades**, invalidating every token derived from it.

💡 Compare to the Unix "root has all rights" model. With capabilities, you say: "here is the right to read *this* exact file, withdrawable at will." It is the opposite of "all or nothing." This idea is not new (it dates to 1966); the project applies it to the runtime of AI agents, with the novelty of **dynamic** delegation/revocation among sub-agents created at runtime.

Commit barrier

A **commit barrier** is a **point of no return**. Once crossed, the actions preceding it can no longer be undone.

Why is it needed? Because some actions are *genuinely* irreversible: once a network packet has left the machine, it cannot be recalled. The barrier marks the boundary between "what can still be undone" and "what is set in stone."

The mechanism is **conservative-hybrid**: - **Automatic** for a short list of provably irreversible effects (a network packet actually sent, a write to an external disk). - **Explicit** (a `commit()` call) for everything else — the normal case. - **Safeguard**: if an agent attempts an external effect without having placed a barrier, the system refuses and suspends the action until the agent decides (commit or undo).

Transaction

A **transaction** is simply the sequence of actions **between two commit barriers**. It is the unit of undo: either all actions in a transaction are undone, or none (never a half-undone state).

Rollback

The **rollback** is the operation that **restores the local state** to what it was at a past instant (but only after the last commit barrier).

⚠️ Rollback is **not** "compensation." It does not call external services back, does not send cancellation messages. It restores the machine's *local* state. Undoing effects that have left for the outside world is an unsolved problem in general; the project explicitly excludes it.

Local state

What rollback can restore: the **local state** = the state of local actors + the local store + messages still in transit *inside* the node. It **excludes** anything that has left for the outside (network, third-party services, other machines).

Content-addressed

A central concept, slightly disorienting at first. Usually, you store data at an *address* you choose ("slot #5"). In **content-addressed** mode, a datum's address **is computed from its content**, via a hash function.

 **Hash:** a function that turns any data into a fixed-size fingerprint (here 32 bytes, using the SHA-256 algorithm). The same data always gives the same fingerprint; two different data give (in practice) different fingerprints. It is like a digital fingerprint of the content.

Remarkable consequences: - The identifier is **deterministic** (content determines the address). - It is **unforgeable** (impossible to craft data matching a fingerprint chosen in advance). - Successive versions do not overwrite each other: each version has its own fingerprint, so **history is preserved for free**. This is precisely what makes rollback possible. (This is also how Git works.)

Asymmetric supervisor

The **human role** in the system. "Asymmetric" because it **is not in the loop on every action**. It has three powers, via privileged capabilities: - **Observe** the full causal log (everything that happened and why). - **Intervene**: revoke capabilities, suspend an agent, force a rollback. - **Authorize**: sign high-impact actions placed on hold.

It has no interactive terminal. Its interface is an external client speaking the same protocol as the agents, but with privileged rights.

5. The properties

Here is the heart of the thesis. The project claims that an OS designed for profile B can **guarantee by construction** properties that Linux offers only at the cost of expensive application layers.

A method rule first: **a property is retained only if it is experimentally verifiable**. A property you cannot measure is an intention, not a property.

There are six properties, P1 to P6. The three "showcase" ones are P1, P2, and P3.

P1 — Density $\times 5$

Statement: host at least **five times more** *dormant* agents than Linux+containers on the same hardware.

💡 A *dormant* agent waits for a message, doing nothing. The question is: how many can you stack in memory before saturating the machine?

Why it is plausible: a Docker container keeps a full Python interpreter in memory even when the agent does nothing (~44 MB/agent). WASM technology (see section 6), by contrast, leaves a dormant agent's pages *virtual* (not loaded into physical memory): ~9.65 KB/agent. The measured ratio is **4,500 to 7,375 $\times$** — far beyond the targeted $\times 5$.

Status: partial. The memory ratio vastly exceeds the target, but it was measured on Linux against Docker, not in a strict comparison on the final target substrate. (See section 11 for the honest nuance.)

P2 — Transactional rollback ≤ 100 ms

Statement: be able to return to any past state (since the last commit barrier) in bounded time, ≤ 100 ms for 100 actions back.

Why it is the main differentiator: without clean rollback, the system brings nothing new — it is merely lighter. With it, an agent can explore, make mistakes, and cleanly back out, which is impossible natively on Linux.

⚠️ **An honest correction by the project:** initially, the specification promised $O(\log N)$ complexity (a cost that grows *very* slowly with the number of actions). Under test, the implementation turned out to be $O(\text{depth})$ — a cost *linear* in the rollback depth. The $O(\log N)$ promise was **withdrawn** (not papered over), because the real guarantee that matters is the **time bound** (≤ 100 ms), which does hold. This kind of owned-up retraction is a hallmark of the project's method.

💡 **$O(\dots)$ notation (algorithmic complexity):** a way to describe how an operation's cost grows with the problem size. $O(N)$ = cost proportional to N (twice the data $\rightarrow$ twice as slow). $O(\log N)$ = cost that grows very slowly (a thousand times more data $\rightarrow$ only about ten times slower). " $O(\text{depth})$ " means the cost depends on the number of steps you walk back, not on the total amount of data.

Status: PASS. Measured: 17–20 ms to roll back 500 actions — five times under target.

P3 — Causal traceability ≤ 10 ms

Statement: retrieve any action by its identifier at **p99 ≤ 10 ms**, even over a log of **10^8 (one hundred million) actions**.

💡 **p99 (99th percentile):** if 99% of lookups are faster than 10 ms, we say "p99 $\leq$ 10 ms." This is stricter than "10 ms on average": it also bounds the slow cases. We measure p99 because a guarantee is only as good as its common worst case, not its average. (A *percentile* is a cutoff: the 99th percentile is the value below which 99% of measurements fall.)

Why it is plausible: retrieving a datum by its key in a well-built index is the most optimized database operation. The project uses RocksDB (see section 6), tailored exactly for this.

Status: PASS on Linux read-only: p99 of **1.4 to 1.9 ms** over 10^8 actions — five to seven times under target.

⚠️ The project is precise about the **scope** of this guarantee. The 10 ms bound holds for an *isolated lookup on a static database* (called **P3a**). The full cycle "write then re-read with durability guarantee" (**P3b**) has a distinct bound (≤ 20 ms). And the multi-agent regime under heavy concurrency (**P3c**) has wider bounds still. Confusing these scopes means claiming more than was measured.

P4 — Capability-based isolation

Statement: every access to a resource requires an explicit capability. No access by default. And delegation respects **attenuation** (a child never has more rights than its parent), along two axes: *permissions* (read-only vs read+write) and *scope* (the whole store vs a single subfolder).

Three conditions must hold *together*: 1. **Authorized accesses succeed** (100%). 2. **Unauthorized accesses fail** (100%, with no possible bypass). 3. **Refusals are audited** (recorded in the causal log).

Status: PASS — including against a real attack (see "confused deputy," section 10).

P5 — Transition determinism

Statement: two copies of the same agent, starting from the same state and fed the same sequence of messages, produce the **same** sequence of emitted messages and the **same** final state.

💡 Why is this valuable? Because it makes bugs **replayable**. If an agent crashed yesterday, you can replay its exact sequence today and observe the crash. On a non-deterministic system, every other bug is irreproducible.

💡 **Deterministic vs stochastic:** a *deterministic* process always gives the same result from the same inputs. A *stochastic* process has a share of randomness (large language models are stochastic: the same question can produce two different answers).

For this to work, every source of randomness (the clock, randomness, the model's stochastic output, external data) must go through **substitutable primitives** — entry points the system can replace in replay mode.

⚠ This is a **conditional** guarantee: it holds *if* the substrate prevents unmediated shared memory (otherwise invisible randomness would creep in). That is why concurrency inside an agent is forbidden.

Status: conditional PASS.

P6 — Crash atomicity

Statement: if the agent is abruptly killed *in the middle* of a transaction, after restart the state is **either** the one before the transaction, **or** the one after the last commit barrier — **never an in-between**.

💡 **Atomicity:** the "all or nothing" property. An atomic operation either happens entirely or not at all, never halfway. Here, applied to an agent's full state: no "half-written" state after a failure.

Status: PASS at the process level. Verified by 40 scenarios where the program is killed at 4 different instants: in 100% of cases, the post-restart state is consistent.

⚠ **An owned-up limit:** "abruptly killed" covers the SIGKILL signal (the immediate, non-negotiable termination of a program by the system), a program crash, or the OOM-killer (which kills a program when RAM is exhausted). It does **not** cover power loss or a kernel crash — in those cases, the operating-system cache is lost. This extension requires a test on real hardware (impossible under emulation) and is documented as a known gap, not hidden.

The priority order: what is sacrificed first?

Properties can conflict (for instance, the finer you trace, the slower you get, so P3 versus P1). An arbitration order is therefore needed. This is decision **ADR-0001**:

P4 > P2 > P3 > P6 > P5 > P1

Read as: "if a property must give way, we give up P1 (density) first, P4 (isolation) last."

Justification of the order: - **P4 (isolation) first:** without access control, hosting stochastic agents is outright dangerous. Non-negotiable. - **P2 (rollback) next:** the functional differentiator. Without it, the system brings nothing. - **P3 (traceability):** its *speed* can be relaxed (10 ms → 100 ms) without killing the project, but not its *correctness*. - **P6 (atomicity):** largely a corollary of P2, rarely in tension. - **P5 (determinism):** valuable but degradable to "best effort." - **P1 (density) last:** a numeric target. Hitting ×4 instead of ×5 does not invalidate the thesis, as long as the rest holds.

💡 **Correctness outranks performance.** That is the philosophy this order captures. A fast but incorrect system is worthless; a correct but slightly less dense system remains useful.

The two "regimes"

An important subtlety to avoid misreading the results:

- **Regime R1 ("Effects")**: P2, P3, P4, P6. Active **everywhere**, including when the AI runs on an external service (the cloud).
- **Regime R2 ("Resources")**: P1, P5. Active **only** when AI inference runs *locally* (the runtime controls the model).

⚠ **Never claim all six properties without naming the regime.** P1 and P5 only make sense if the runtime controls how the AI runs. If the AI is called remotely, the runtime controls neither the density nor the determinism of that part.

6. Architecture

Here is how the pieces fit, from top (the agent) to bottom (the hardware):

```
Agent (a .wasm module)
| expresses itself via an ABI = "host functions"
| (agent_infer, emit, agent_add_cause, ...)
▼
Rust / Tokio runtime                                ← the heart of the project (poc/runtime/)
├ Scheduler ..... distributes inference capacity (bounded pool)
├ CausalLog ..... the action log (RocksDB)
├ ContentStore ..... the saved states (RocksDB, Merkle DAG)
└ Capabilities ..... the rights tree (delegation, revocation)
▼
Substrate: Linux (current) · seL4 (target)
```

Let us break down each brick and each technology.

WebAssembly (WASM) and Wasmtime — agent isolation

WebAssembly (WASM) is a compiled, portable program format confined in a **sandbox**: a WASM program runs inside a "bubble" that can touch only what it is explicitly authorized to touch. Originally designed to run code safely in web browsers.

Wasmtime is the engine that executes these WASM programs outside a browser.

Why WASM for agents? Two reasons: 1. **Lightness** (P1): a dormant WASM agent costs ~9.65 KB, versus ~44 MB for a Docker+Python container. This is the foundation of the ×5 density. 2. **Isolation** (P4): the WASM bubble can call only the functions the runtime exposes — a natural fit for the "non-ambient" capability model.

💡 **ABI and host functions:** the agent (in its WASM bubble) cannot call Linux. It can only call the functions the runtime hands it through a precise opening in the bubble: these are the **host functions**. The list of those functions and their format is the **ABI** (Application Binary Interface). Examples: `agent_infer` ("run an AI inference"), `emit` ("publish a message / place a barrier"), `agent_self_rollback` ("go back").

Tokio — the asynchronous engine

Tokio is a Rust library for writing **asynchronous** code: managing thousands of waiting tasks (for a message, a response) without blocking, using very few system threads. This is what makes hosting many dormant agents cheap.

Rust — the language

The whole runtime is written in **Rust**, a language that guarantees at compile time the absence of a whole class of memory bugs (out-of-bounds access, use-after-free) **without a garbage collector**. For an OS-class component that must be both safe and fast, it is the reference choice today.

RocksDB — the storage engine

RocksDB is a very fast, embedded (no separate server) key-value database. The project uses it for **two** distinct databases: the CausalLog and the ContentStore.

Why RocksDB rather than SQLite? Because RocksDB is built on an **LSM tree**, the right tool for this need:

💡 **LSM tree (Log-Structured Merge tree):** a data structure optimized to *write a lot, fast* (always appending at the end, never in the middle), then *merge* in the background — an operation called **compaction**. By contrast, a **B-tree** (used by SQLite) is optimized for scattered reads/writes. For a log that only appends 10^8 entries and re-reads them by key, the LSM tree is architecturally the right choice.

💡 **Bloom filter:** a small probabilistic filter that very quickly answers "is this key *certainly absent*?" If it says "absent," there is no need to read the disk. This is what makes lookups of nonexistent keys nearly free, and helps hold P3's p99.

CausalLog — the action log

The **CausalLog** (`poc/causal-log/`) is the **append-only** (only appended, never modified) log of all actions.

- **Key** = the `action_id` = the SHA-256 fingerprint of the entry (hence content-addressed).

- **Value** = the LogEntry: { agent_id, sequence number, timestamp, parent_ids[], post-state fingerprint, payload }.

The crucial field is **parent_ids[]**: the **list** of actions that directly caused this one. This is what forms the **causal DAG** (see Bet 1, section 8).

ContentStore — the saved states

The **ContentStore** (poc/store/) keeps **snapshots** of agents' state, as a **Merkle DAG**.

💡 **Merkle DAG**: a graph where each node is addressed by the fingerprint of its content, and each node references its parents by their fingerprint. Consequence: to go back, you just walk the chain `parent → parent → ...` up to the target. This is exactly Git's structure. It is what makes rollback (P2) feasible without copying the whole state each time.

Each snapshot is a SnapshotHeader: { data_hash, parent (optional), seq, ts }. Rollback walks this chain, one RocksDB read per link (hence $O(\text{depth})$).

💡 **The "no-force" discipline** (ADR-0027): the ContentStore may run *ahead* of the log (a saved state the log does not yet reference — this is an "orphan," harmless, garbage to be collected later). But it must **never lag behind** (the log referencing a state absent from the store — that is corruption, a "dangling reference"). This asymmetry is what makes P6 tenable without paying a costly disk sync on every action.

💡 **Forced disk sync (fsync)**: a command that forces the system to *actually* write to disk what was still only in an in-memory cache. It is slow but is the only guarantee that data will survive a power loss. The "no-force" discipline is precisely about *avoiding* this cost on the common path, relying on the operating-system cache, which survives a mere program kill.

Capabilities — the rights tree

The **Capabilities** module (poc/capabilities/) maintains the delegation tree: who gave which (reduced) right to whom. Revoking a node recursively invalidates its entire subtree — hence the $O(\text{depth})$ cost of revocation.

Scheduler and inference pool — sharing the AI

Inference (running the language model to produce a response) is a **scarce, costly resource**: on a central processor (CPU), one cycle takes 6 to 18 seconds. Several agents share it. The **scheduler** manages an **inference pool**: a semaphore of capacity k (only k inferences at a time).

💡 **Semaphore:** a token counter. To infer, an agent must take a token; it returns it when done. If no token is left, it waits. This is what *bounds* the number of simultaneous inferences.

Two important guarantees: - **Priority:** `Foreground` goes before `Batch` (background). - **Anti-starvation (fairness):** a `Batch` job that waits too long is promoted to `Foreground`, so it is never forgotten indefinitely. (Without this, a stream of high-priority jobs would starve background jobs forever.)

7. The journey of one action

Let us assemble the pieces. Here, simplified, is what happens when an agent processes a message (the "W1 cycle" used in benchmarks):

1. **Reception.** A message arrives in the agent's inbox (the Tokio queue). ← *this is an action (reception).*
2. **Introspection.** The agent calls `agent_introspect()` to read its current state (sequence number, lifecycle).
3. **Inference.** The agent calls `agent_infer(prompt)`. The scheduler grants it a token from the inference pool (or makes it wait). The model thinks.
4. **Commit barrier.** The agent calls `commit()`: it places a point of no return. From here on, everything before becomes non-undoable.
5. **Emission.** The agent calls `emit(message)` to publish a result. ← *this is an action (emission).*

On every action, **behind the scenes:** - a `LogEntry` is appended to the **CausalLog** (with its `action_id` = fingerprint, its `parent_ids`, the post-state fingerprint) → this feeds P3 (traceability); - a snapshot may be written to the **ContentStore** → this feeds P2 (rollback); - every resource access is checked against a **capability** → this feeds P4 (isolation); - every source of randomness goes through **substitutable primitives** → this feeds P5 (determinism).

And if the process is **abruptly killed** between steps 4 and 5? On restart, RocksDB replays its write-ahead log (WAL — the ledger where each change is noted *before* being applied), and the state returns either to "before the transaction" or to "after the last commit barrier" — never the middle. → this is P6 (atomicity).

💡 **The takeaway:** the six properties are not six separate modules. They emerge from **the same shared infrastructure** (content-addressed log + Merkle store + capability tree + substitutable primitives). That is why adding P4, P5, and P6 to P1, P2, and P3 does not triple the cost: everything is shared. (This is the "synergies" of the specification.)

8. The five bets

The project rests on five architectural bets. Each is **falsifiable**: the condition that would refute it was written *before* the experiment.

Bet 1 — A causal DAG, not a tree

💡 **DAG** = Directed Acyclic Graph. A **tree** is the special case where each node has *only one* parent. A **DAG** is more general: a node can have *several* parents. ("Acyclic" means you cannot return to your starting point by following the arrows.)

The bet: real causality between parallel agent actions is a DAG, not a tree. When two sub-agents work in parallel and their results merge, the merge action has **two** parents. A tree (one parent) would force artificial serialization, lying about the true causality.

That is why each action carries a list of parents (`caused_by[]` / `parent_ids[]`). **Validated** from the first multi-agent experiments.

Bet 2 — RocksDB (LSM), not SQLite (B-tree)

The bet: for an append-only log of 10^8 entries with lookup by opaque key, the LSM tree is the right tool (see section 6). **Validated:** $p99 \leq 2$ ms over 10^8 entries, five to seven times under target.

Bet 3 — Wasmtime + Tokio, not Docker

The bet: WASM-module isolation is thousands of times lighter per dormant agent than Docker+Python. **Validated:** 4,500 to 7,375 times less RAM.

The cost was even *modeled*: $\text{overhead}(N) = 9.65 - 54/N$ KB per agent (with goodness of fit $R^2=0.988$). Translation: the asymptotic cost is ~ 9.65 KB/agent; the $54/N$ term represents shared fixed costs (the WASM binary, the Tokio engine) that amortize from $N \geq 300$ agents.

💡 **R^2 (coefficient of determination):** an indicator between 0 and 1 telling how well a formula fits the measured data. 0.988 = the formula explains 98.8% of the observed variation: excellent. ("Asymptotic" is the value approached as N becomes very large.)

Bet 4 — Revocable capabilities, not Unix identity

The bet: delegating precise, revocable rights tokens beats the "root has everything" model. **Validated** functionally, including against the "confused deputy" attack (section 10).

Bet 5 — Asymmetric supervision, not real-time

The bet: the human supervisor observes, intervenes, authorizes — but **not in real time**. This sizes everything else: P3's latency target is 10 ms ("a human can wait"), not 1 μ s ("a hardware interrupt cannot wait"). Corollary: the log is designed **durability-first**, not latency-first.

9. The substrates

The project was built on **two** successive substrates. Understanding why is essential.

💡 **Substrate:** the execution layer *beneath* the runtime — the real kernel everything rests on.

Linux (current substrate)

All functional validation (phases 5 to 7) was done on Linux. The primitives there are real and measured. **But** isolation there is purely *software* (the Wasmtime sandbox). If someone finds a flaw in Wasmtime, they escape the bubble — and on Linux all agents share the same system process, so one escape compromises everything.

⚠️ Measurements made on Linux are valid **on Linux**, but **not transferable** to the target substrate. This is an owned-up decision, not an oversight: measuring density on Linux says nothing about density on seL4.

seL4 (target substrate)

seL4 is a **formally verified microkernel**. Two notions to decode:

💡 **Microkernel:** a minimalist kernel that does *only* the strict minimum (memory management, inter-process communication), leaving the rest to user-space programs. Smaller = easier to verify.

💡 **Formally verified:** it has been **mathematically proven** (the proof itself machine-checked, not merely "tested") that the kernel's code conforms to its specification. seL4 is one of the very few kernels in the world in this situation. This makes it a trust base of an entirely different level than Linux (~30 million unproven lines).

💡 **Trusted Computing Base (TCB):** the set of code you *must* trust for security to hold. The smaller and more proven, the better. The project's whole seL4 argument is about shrinking and proving this base.

What was ported to seL4 (11 milestones, C.1 to C.11, on the QEMU emulator for a 64-bit ARM processor, called AArch64):

💡 **QEMU:** an emulator — software that simulates another computer (here, an ARM machine) on the development machine. Handy for testing without physical hardware, but unsuitable for measuring real disk-access speeds.

💡 **virtio-blk:** a standardized *virtual* disk the emulator provides to the guest system. The seL4 store writes to it as to a real disk.

- Running Wasmtime *without a standard library* (`no_std` in Rust — a real technical challenge, since most code assumes a full operating system beneath it).
- A persistent store (redb, a key-value database, writing to the virtio-blk virtual disk).
- Enforcing the **W^X** principle on the just-in-time compiler (see below).
- Multi-agent commit with a per-agent capability "badge."
- Crash atomicity under adversarial kill scenarios.
- Isolation proven against **malicious** WASM modules (attempting out-of-bounds access, division by zero, infinite loop).

💡 **W^X (Write XOR eXecute):** a memory region is either *writable* or *executable*, never both at once. This prevents an attacker from writing code and then executing it. It is tricky for a **just-in-time** compiler (JIT — which generates machine code during execution), because it must precisely write *then* execute — hence the controlled permission switch at the right moment.

The seL4 prototype was **closed** (ADR-0049) once these milestones were reached.

10. The method

What distinguishes this project from a mere prototype is its **methodological discipline**. Five commitments.

Decisions are explicit and versioned (the ADRs)

💡 **ADR** = Architecture Decision Record. Each important decision is recorded in a numbered file (`decisions/0001-...`, `0002-...`) stating: *what* was decided, *why*, *which alternatives* were rejected, and *under what conditions* it should be revisited.

There are 56 of them (in `decisions/`). An ADR is **binding**: until amended or replaced, it must be followed. A prototype that deviates from one is a **debt to track**, not a precedent to follow.

💡 Why is this powerful? When an experimental result contradicted a hypothesis (for example a false memory-leak alarm), the ADR system provided the framework to decide rationally — refute the hypothesis, amend the design, or adjust the measurement. Nothing is buried quietly.

Hypotheses are written *before* the experiment

Each benchmark has a **target defined in advance**. A result that beats the target by 17× is meaningful only *because* the target was fixed beforehand. Setting the target afterward would prove nothing.

Failures are documented, not buried

Three "**refutation ADRs**" (0032, 0033, 0034) document cases where the data contradicted the initial model. Example: an endurance test ("T6-soak") seemed to show a memory leak (RAM climbing). Investigation: the statistical criterion used (a linear regression) was structurally inapplicable because of RocksDB compaction spikes ($R^2=0.24$, so the fit meant nothing). The leak hypothesis was **explicitly refuted**: memory is in fact bounded. The right response was to fix the model, not discard the data.

"Wrong-substrate" measurements are refused

A measurement that proves nothing about the target is not a useful measurement. Measuring P3a on QEMU's emulated storage would say nothing about real latency on actual seL4 hardware. The project therefore **refused** this measurement (it awaits physical hardware) rather than publish a misleading number. It cost one measurement, and gained integrity.

We attack the system, not merely validate it

💡 "A property that only holds under friendly inputs is not a property."

The system was subjected to **adversarial campaigns** (ADR-0050/0051): we attack, we do not merely check. The emblematic example is the "**confused deputy**":

💡 **Confused deputy**: a classic attack where a malicious actor, lacking the rights, manipulates a *legitimate* intermediary (which does hold the rights) into performing the forbidden action on its behalf. The deputy is "confused": it acts with its own rights, unaware it is serving an attacker.

Result: isolation **held** (the attack failed to escalate privileges). **But** the campaign revealed an *audit gap*: by flooding the system with more than 100 benign refusals per second, an attacker could *mask* a malicious refusal in the noise. This was not an isolation flaw (the right remained correctly refused), but an **observability** flaw. The gap was **fixed** (aggregating refusals per resource) and the fix **re-tested** with a second attack. This is exactly the intended cycle: attack → find → fix → re-test.

11. The balance sheet

Honesty about what is achieved and what is not is a core value of the project.

What is proven

Property	Status	Evidence
P2 Rollback	✓ PASS	17–20 ms for depth=500 (target ≤ 100 ms)
P3a Traceability (lookup)	✓ PASS (Linux, read-only)	p99 1.4–1.9 ms over 10^8 actions
P4 Isolation	✓ PASS	holds under the confused-deputy attack
P5 Determinism	✓ PASS (conditional)	replay of 1,000 messages identical
P6 Crash atomicity	✓ PASS (process level)	40 kill scenarios, 100% consistent
seL4 integration	✓ C.1–C.11	11 milestones on QEMU AArch64 emulator

What is NOT proven (and why, owned up)

- **P1 quantified vs Linux+containers (strictly)**: the memory ratio vastly exceeds the target ($\times 4,500$ and up), but the strict "at equivalent action latency on the target substrate" comparison was not established — because the Linux figures are **not transferable** to seL4. *Explicit decision, not a missing measurement.*
- **P3a on real seL4 hardware ("D-P3a")**: the measurement bench is ready, but it needs a **physical ARM board** (or direct disk access, NVMe passthrough) — QEMU's emulated disk access is not an acceptable measurement substrate. *Hardware-gated.*
- **Power-loss durability**: out of scope. The consistency oracle is written, awaiting the same hardware trigger.
- **Cross-store atomicity**: a narrow window remains where, under cache loss, the log and the store (two separate RocksDB databases) could diverge. Mitigated by a fail-safe at restore time (which detects the problem), not closed. Closing it (atomic commit across the two) is tied to the future garbage-collection work.

💡 The methodological lesson of this balance sheet: **"not proven" is not "failed."** The project rigorously distinguishes *measured*, *planned*, *deferred for hardware reasons*, and *abandoned because non-transferable*. Conflating these four statuses would amount to lying about the real state.

12. Pocket glossary

Term	One-line definition
Operating system (OS)	Program that arbitrates programs' access to hardware
Kernel	The all-powerful heart of the operating system
Runtime	Program that hosts and serves other programs (here: the agents)
Agent	AI-driven, autonomous program identified by a permanent <code>agent_id</code>
Actor	Entity that receives/processes/emits messages, one at a time
Action	A message received or emitted — the project's basic unit
Capability	A precise rights token: non-ambient, delegable, attenuable, revocable
Attenuation	A derived capability has <i>at most</i> its parent's rights
Commit barrier	Point of no return; after it, no more rollback
Transaction	Sequence of actions between two commit barriers (all or nothing)
Rollback	Restoration of the local state to a past instant
Local state	Local actors + local store + internal messages (excludes the external)
Content-addressed	Data stored at an address = the fingerprint of its content
Hash (SHA-256)	Fixed-size fingerprint of a datum (32 bytes here)
DAG	Directed acyclic graph; a node can have several parents
Merkle DAG	DAG where each node is addressed by its content's fingerprint (cf. Git)
Snapshot	Photograph of an agent's state at a given instant
CausalLog	The append-only action log (RocksDB)
ContentStore	The store of state snapshots (RocksDB, Merkle DAG)

Term	One-line definition
WASM / WebAssembly	Portable program format, confined in a sandbox
Sandbox	Isolated environment where a program touches only the allowed
Wasmtime	Engine that runs WASM outside a browser
ABI / host function	The functions the runtime exposes to the agent in its bubble
Tokio	Rust library for asynchronous code (many waiting tasks)
RocksDB	Embedded, fast key-value database (LSM tree)
LSM tree	Structure optimized to write a lot and merge in the background
B-tree	Structure optimized for scattered access (SQLite) — not chosen here
Compaction	Background merging of an LSM tree's writes
Bloom filter	Filter that quickly says whether a key is certainly absent
Scheduler	Component that distributes inference capacity among agents
Inference pool	Semaphore bounding the number of simultaneous AI inferences
Semaphore	A token counter for access to a limited resource
Inference	Running the language model to produce a response (a scarce resource)
Disk sync (fsync)	Forcing the actual write to disk (slow, but safe against power loss)
Write-ahead log (WAL)	Ledger where each change is noted before being applied
p99 / percentile	99th percentile: 99% of cases are faster than this value
O(...) complexity	A way to describe how a cost grows with problem size
Deterministic / stochastic	Always the same result / has a share of randomness
Atomicity	The "all or nothing" property: entirely, or not at all
Asymmetric supervisor	The human role: observes, intervenes, authorizes — not real-time

Term	One-line definition
Profile B	The target agent: 1 h – 1 month, persistent state, 10^4 – 10^8 actions, occasional supervision
Substrate	The execution layer beneath the runtime (Linux or seL4)
seL4	Formally verified microkernel (target substrate)
Microkernel	Minimalist kernel, easier to prove
Formally verified	Mathematically proven to conform to its specification (not merely tested)
Trusted Computing Base (TCB)	The code that must be trusted
W^X	Memory either writable or executable, never both
Just-in-time (JIT)	Generation of machine code during execution
no_std	Rust code without a standard library (required on seL4)
QEMU	Emulator that simulates another machine (here ARM)
AArch64	64-bit ARM processor architecture (the seL4 target)
virtio-blk	Standardized virtual disk provided by the emulator
ADR	Architecture Decision Record (what, why, alternatives, conditions)
Confused deputy	Attack manipulating a legitimate intermediary for its rights
no-force	Discipline: the store may run ahead of the log, never behind
Regime R1 / R2	R1 (effects, everywhere) / R2 (resources, local inference only)

Going further

For...	See...
The vision and problem in detail	<code>spec/01-vision.md</code>

For...

See...

The formal definition of the properties

[spec/02-properties.md](#)

The full, nuanced glossary

[spec/06-glossary.md](#)

The executive summary

[OVERVIEW.md](#)

The condensed technical view

[wiki/01-README-technique.md](#)

All architecture decisions

[decisions/INDEX.md](#) (56 ADRs)

The empirical lessons

[lab/LESSONS.md](#) (L1–L119)

Project status and debts

[TODO.md](#)

The French edition of this guide

[docs/guide-apprentissage.md](#)

...